Gravitee

Platform Engineer — Technical Exercise

Build a minimal Gravitee API Management stack on a local Kubernetes cluster so that `GET /ping` returns `pong` through the gateway.

Plan on 3 to 4 hours of work; you have 5 working days to send it back.

The task

1. Create a local Kubernetes cluster with the tool of your choice.
2. Deploy Gravitee APIM (Community Edition) and whatever the gateway needs to get its API configuration.
3. Deploy a small backend service that replies pong. Any language or image is fine.
4. Expose `GET /ping` on the Gravitee gateway, proxied to your backend. The response body must be `pong`.

The official Gravitee documentation is a good starting point. How you use it is up to you.

Requirements

1. **One command to start, one to tear down.** From a clean clone, a single command builds everything and another removes it. Use whatever entry point you prefer. The only prerequisites are the tools listed in your README.
2. **Configuration as code.** Everything you set up is defined in code, with no manual steps. Running it again should give the same result.
3. **Production-grade configuration.** Write the deployment configuration to the standard you'd apply to a production system you're handing over to a colleague. Don't deploy anything to a cloud provider. Everything runs on your local cluster, and the configuration is what has to be production-ready.
4. **Portable.** We will test your solution on macOS and Linux machines.

5. **Your assumptions, written down.** If something in this brief is unclear, make a reasonable choice and explain it in the README.

6. **A README that covers:**

   - prerequisites
   - how to run it and how to test `/ping`
   - the architecture (a diagram is welcome)
   - the choices you made and the alternatives you considered
   - what's missing before this could run in production, and what you would do next

Optional

- **Elasticsearch and analytics are not required.** If your machine struggles with them, leave them out and say so in the README.
- **SSL certificate auto-renewal.** Serve the API over HTTPS with certificates that are issued and renewed automatically.
- **Metrics endpoint.** Expose the gateway's metrics so a monitoring system could scrape them, and say in the README what you would watch.
- Anything beyond the requirements is optional. A small, clean setup with a clear README beats a big, unfinished one.

Rules

- **Time box:** aim for 3 to 4 hours of work. That is deliberately tight, so we expect trade-offs. If you run out of time, stop and write in the README what you dropped, why, and what you would have done next. That counts too.
- **AI tools are welcome.** Use whatever you'd use at work. Be ready to explain and change any line of it.
- **What to send:** a link to a private Git repository. We'll send you the accounts to grant access to. Keep the full commit history; don't squash it.
- **What it costs to run.** For reference, our own version of this fits in 4 CPUs and 6 GB of memory given to Docker, and pulls around 2 GB of images the first time.

---

https://documentation.gravitee.io